



Artificial Intelligence Engineering Department

TERM PROJECT REPORT

Machine Learning Lifecycle Management System Using MLflow

Course	AIN-3009 Machine Learning Operations (MLOps)
Instructor	Prof. Murat Gosin Bakir
Department	Artificial Intelligence Engineering
Student Name	Mourad sleem
Student Number	2106045
Date	June 2026

1. Introduction & Domain Selection

This project implements a state-of-the-art Machine Learning Operations (MLOps) pipeline designed to manage the full lifecycle of a predictive model.

Domain & Dataset

The chosen domain is the Financial Sector, specifically focusing on **Credit Card Fraud Detection**. The dataset utilized is the Kaggle Credit Card Fraud Detection dataset comprising 284,807 transactions. Due to extreme class imbalance — only approximately 0.17% of transactions are fraudulent — robust evaluation metrics and preprocessing techniques including SMOTE and synthetic data augmentation were implemented.

Managing the lifecycle of a model in this high-stakes domain requires rigorous experiment tracking, hyperparameter tuning, model governance, and continuous monitoring. MLflow acts as the central orchestrator for all these requirements.

PROJECT FLOW — Overall MLOps Project Pipeline

Step 1 Data Ingestion & Preprocessing (SMOTE, Feature Engineering)



Step 2 Experiment Tracking — MLflow Tracking Server (6 Baseline Models)



Step 3 Hyperparameter Tuning — Optuna Bayesian Optimization (50 Trials)



Step 4 Model Registration — MLflow Model Registry (FraudDetector)



Step 5 Model Deployment — FastAPI REST API (Production Stage)



Step 6 Drift Monitoring — Evidently AI + Auto-Retraining Trigger

2. Objective 1 — Experiment Tracking with MLflow

MLflow Tracking is integrated deeply into the training pipeline (src/train.py). Six different baseline algorithms are trained and comprehensively tracked: Logistic Regression, Random Forest, XGBoost, LightGBM, CatBoost, and an MLP Neural Network.

Tracked Metrics & Artifacts

- ◆ Hyperparameters: Model-specific configurations (max_depth, learning_rate, C, etc.)

- ◆ Advanced Evaluation Metrics: Matthews Correlation Coefficient (MCC), Brier Score Loss, and Log Loss — all tailored for imbalanced data.
- ◆ Artifacts: Confusion Matrices, ROC/PR Curves, Feature Importance plots, and the exact config.yaml for full reproducibility.
- ◆ System Metrics: Real-time CPU, GPU, RAM, and Disk I/O utilization logged during training.
- ◆ Rich Tagging: Runs tagged by task and framework, with Markdown descriptions rendered in the MLflow UI.

PROJECT FLOW — Experiment Tracking Workflow

Step 1 Load config.yaml (hyperparameter definitions)



Step 2 Train each of 6 algorithms (src/train.py)



Step 3 Log parameters, metrics & system usage to MLflow Run



Step 4 Upload visual artifacts (Confusion Matrix, ROC, Feature Importance)



Step 5 Tag run with task & framework metadata



Step 6 Compare all 6 runs on the MLflow Tracking Dashboard

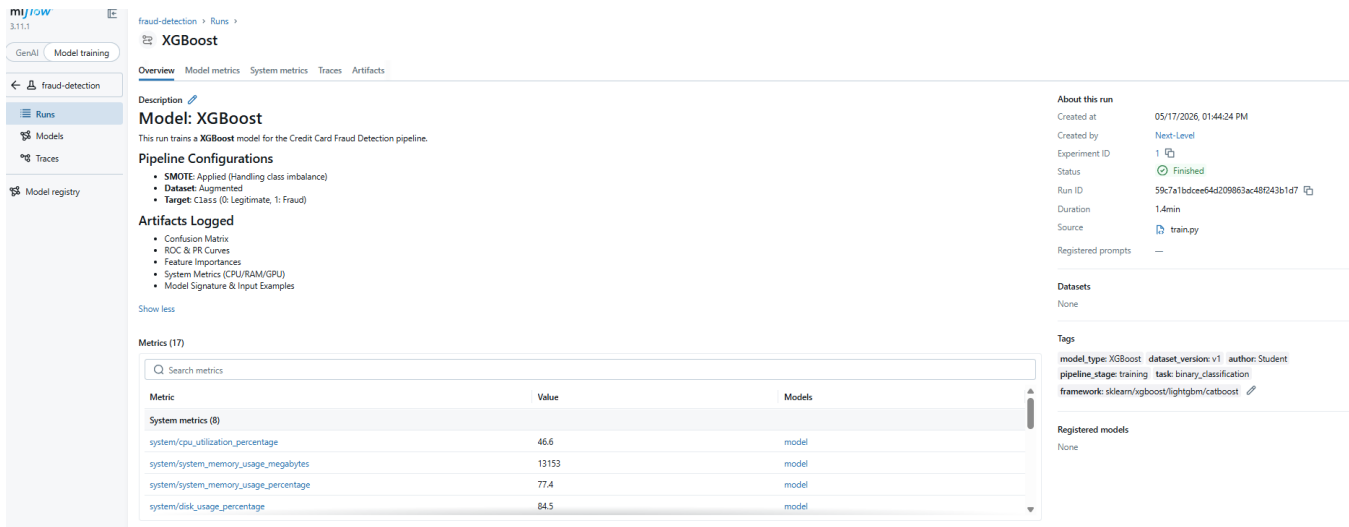
Evidence 1: MLflow Experiment Tracking Dashboard

All 6 baseline models tracked with hyperparameters, MCC/Brier metrics, and rich run tags.

Run Name	Created	Dataset	Duration	Source	Models
Optuna_XGBoost_Tu...	2 minutes ago	-	-	tune.py	-
Stacking_Ensemble	18 minutes ago	-	15.7min	stacking...	model
MLP_Neural_Network	24 minutes ago	-	6.1min	train.py	model
CatBoost	27 minutes ago	-	2.2min	train.py	model
LightGBM	29 minutes ago	-	2.2min	train.py	model
XGBoost	30 minutes ago	-	1.4min	train.py	model
Random_Forest	36 minutes ago	-	5.9min	train.py	model
Logistic_Regression	39 minutes ago	-	2.5min	train.py	model
Auto_Retrain_XGBoost	11 days ago	-	32.4s	auto_ret...	model
Monitoring_Pipeline	11 days ago	-	4.2s	monitor...	-
SHAP_Explainability	11 days ago	-	1.5s	explaina...	-
Auto_Retrain_XGBoost	11 days ago	-	37.2s	auto_ret...	model
Monitoring_Pipeline	11 days ago	-	4.2s	monitor...	-
SHAP_Explainability	11 days ago	-	1.5s	explaina...	-
Threshold_Optimizati...	11 days ago	-	348ms	threshol...	-
Auto_Retrain_XGBoost	11 days ago	-	39.7s	auto_ret...	model
Monitoring_Pipeline	11 days ago	-	4.8s	monitor...	-
Stacking_Ensemble	11 days ago	-	15.7min	stacking...	model
Optuna_XGBoost_Tu...	11 days ago	-	6.7min	tune.py	FraudDetector v3

Evidence 2: System Metrics & Run Details

Detailed run page showing the injected Markdown description and live hardware usage.



3. Objective 2 — Hyperparameter Tuning with Optuna

To elevate predictive performance, hyperparameter tuning is orchestrated using Optuna (src/tune.py). Bayesian Optimization is performed over 50 trials to find the optimal XGBoost configuration.

- ◆ Each Optuna trial is logged as a nested run inside the MLflow Tracking server.
- ◆ MLflow captures the objective value (maximizing F1-Score / MCC) at every step.
- ◆ The best hyperparameter configuration is automatically isolated and used for the final model build.

PROJECT FLOW — Hyperparameter Tuning Flow

Step 1 Define search space (max_depth, learning_rate, n_estimators, etc.)



Step 2 Optuna Bayesian Optimization — Trial N of 50



Step 3 Log trial as nested MLflow Run (params + MCC objective value)



Step 4 Select best trial — save optimal config



Step 5 Retrain final XGBoost model on full training set

Evidence 3: Hyperparameter Tuning — Optuna Nested Runs

Nested MLflow runs generated during 50 Optuna trials for the XGBoost model.

fraud-detection > Runs >

Optuna_XGBoost_Tuning

Overview Model metrics System metrics Traces Artifacts

About this run

Created at	05/17/2026, 02:12:49 PM
Created by	Next-Level
Experiment ID	1 
Status	 Running
Run ID	d6201f61c7f0424ba5c724de8b575977 
Duration	
Child runs	trial-46 , trial-45 , trial-44 , trial-43 , trial-42 , trial-41 , trial-40 , trial-39 , trial-38 , trial-37
	Load more
Source	 tune.py
Logged models	
Registered prompts	—

4. Objective 5 — Model Registry & Lifecycle Management

Model governance is strictly enforced using the MLflow Model Registry (src/register_model.py). Structured lifecycle stages control which model version is exposed to downstream consumers.

- ◆ Model Signatures: mlflow.models.infer_signature captures the exact input schema (features and data types).
- ◆ Input Examples: A sample of training data is injected directly into the registry for validation and testing.
- ◆ Stage Transitions: The model FraudDetector transitions through None → Staging → Production in a controlled, auditable manner.

PROJECT FLOW — Model Registry Lifecycle

Step 1 Best model identified from tuning & experiment tracking



Step 2 Register model as FraudDetector in MLflow Registry (None stage)



Step 3 Validate model in Staging environment (schema & performance checks)



Step 4 Promote to Production stage — available for live deployment



Step 5 Archive older model versions automatically

Evidence 4: MLflow Model Registry Lifecycle

Model Registry interface showing FraudDetector versions transitioning into Staging and Production.

Registered Models >

FraudDetector

Created Time: 05/05/2026, 01:37:03 PM

Last Modified: 05/17/2026, 02:20:11 PM

▼ Description [Edit](#)

XGBoost classifier for credit card fraud detection. Trained on the Kaggle Credit Card Fraud dataset with Optuna-tuned hyperparameters and SMOTE oversampling.

> Tags

▼ Versions All Active 1 Compare

New model registry UI

Version	Registered at ↕	Created by	Stage	Description
✓ Version 4	05/17/2026, 02:20:10 PM		Production	Version 4 — Best model from O...
✓ Version 3	05/05/2026, 06:30:43 PM		Archived	Version 3 — Best model from O...
✓ Version 2	05/05/2026, 01:38:55 PM		Archived	Version 2 — Best model from O...
✓ Version 1	05/05/2026, 01:38:14 PM		None	Version 1 — Best model from O...

Evidence 5: Model Signature & Input Schema

Recorded Model Signature and input example proving deployment schema awareness.

The screenshot displays the MLflow Model Registry interface. On the left, a sidebar shows the navigation menu with options like 'Runs', 'Models', 'Traces', and 'Model registry'. The main panel shows the details of a model named 'MLmodel' (4.64KB). The model is listed as 'Not registered'. Below the model name, there is a list of artifacts: 'conda.yaml', 'input_example.json', 'model.pkl', 'python_env.yaml', 'requirements.txt', and 'serving_input_example.json'. The 'Artifacts' tab is selected, showing the model's metadata and signature. The metadata includes the path to the model, artifact path, flavors (python_function, env, sklearn), and various identifiers like mlflow_version, model_id, model_size_bytes, model_uuid, and run_id. The signature section shows the input schema, which is a list of 17 features (V1 to V17) with their respective types and required status.

```
Path: file:C:/1BAU/8_SEMESTER/MLOPS/Term_project/mlruns/1/models/m-5f5d64202305461caf096f5b3e0bdc56/artifacts/MLmodel
artifact_path: file:C:/1BAU/8_SEMESTER/MLOPS/Term_project/mlruns/1/models/m-5f5d64202305461caf096f5b3e0bdc56/artifacts
flavors:
  python_function:
    env:
      conda: conda.yaml
      virtualenv: python_env.yaml
    loader_module: mlflow.sklearn
    model_path: model.pkl
    predict_fn: predict
    python_version: 3.13.1
  sklearn:
    code: null
    pickled_model: model.pkl
    serialization_format: cloudpickle
    sklearn_version: 1.8.0
    skops_trusted_types: null
is_signature_from_type_hint: false
mlflow_version: 3.11.1
model_id: m-5f5d64202305461caf096f5b3e0bdc56
model_size_bytes: 8732583
model_uuid: m-5f5d64202305461caf096f5b3e0bdc56
prompts: null
run_id: 2a4572e6024e424fac73250de7a0c480
saved_input_example_info:
  artifact_path: input_example.json
  pandas_orient: split
  serving_input_path: serving_input_example.json
  type: dataframe
signature:
  inputs: [{"type": "double", "name": "V1", "required": true}, {"type": "double", "name": "V2", "required": true}, {"type": "double", "name": "V3", "required": true}, {"type": "double", "name": "V4", "required": true}, {"type": "double", "name": "V5", "required": true}, {"type": "double", "name": "V6", "required": true}, {"type": "double", "name": "V7", "required": true}, {"type": "double", "name": "V8", "required": true}, {"type": "double", "name": "V9", "required": true}, {"type": "double", "name": "V10", "required": true}, {"type": "double", "name": "V11", "required": true}, {"type": "double", "name": "V12", "required": true}, {"type": "double", "name": "V13", "required": true}, {"type": "double", "name": "V14", "required": true}, {"type": "double", "name": "V15", "required": true}, {"type": "double", "name": "V16", "required": true}, {"type": "double", "name": "V17", "required": true}, {"type": "double", "name": "V18", "required": true}]
```

5. Objective 3 — Model Deployment

The deployment stage (src/serve.py) bridges the Model Registry with live production environments through a high-performance FastAPI REST server.

- ◆ Upon startup, the API dynamically fetches the active model via URI: `models:/FraudDetector/Production`.
- ◆ Decoupled architecture: promoting a new model in MLflow automatically updates the inference engine without changing the API codebase.
- ◆ Exposed endpoints: `/health` (health check), `/model-info` (metadata), `/predict` (single & batch predictions).

PROJECT FLOW — Deployment Pipeline

Step 1 Model promoted to Production in MLflow Registry



Step 2 FastAPI server starts — fetches models:/FraudDetector/Production



Step 3 Client sends POST request to /predict (JSON payload)



Step 4 API preprocesses input — runs inference via loaded model



Step 5 Returns fraud probability + binary classification label

Evidence 6: FastAPI Deployment Server

FastAPI Swagger (/docs) running the models:/FraudDetector/Production inference engine.

Fraud Detection API 1.0.0 OAS 3.1

/openapi.json

Real-time credit card fraud detection powered by an XGBoost model managed through MLflow. Part of the AIN-3009 MLOps Term Project.

General



GET / Root



GET /health Health



GET /model-info Model Info



Predictions



POST /predict Predict



POST /predict_batch Predict Batch



Schemas



BatchPredictionResponse > Expand all object

BatchTransactions > Expand all object

HTTPValidationError > Expand all object

HealthResponse > Expand all object

ModelInfoResponse > Expand all object

6. Objective 4 — Performance Monitoring & Drift Detection

Financial ML models are highly susceptible to concept drift as fraudster behaviour evolves over time. A continuous monitoring pipeline detects distributional shifts and triggers automatic retraining.

- ◆ **Monitoring Pipeline:** `src/monitor.py` simulates batches of incoming real-time transaction data.
- ◆ **Evidently AI:** Incoming data is continuously compared against the original training distribution to detect drift.
- ◆ **Drift Artifacts:** Drift scores and comprehensive HTML drift reports are logged directly into MLflow as artifacts.
- ◆ **Auto-Retraining:** If drift exceeds the threshold configured in `config.yaml`, `src/auto_retrain.py` retrains a fresh model and promotes it to the Registry.

PROJECT FLOW — Monitoring & Auto-Retraining Flow

Step 1 Batch of incoming transactions received (`src/monitor.py`)



Step 2 Evidently AI computes drift score vs. original training distribution



Step 3 Drift report generated — logged as HTML artifact in MLflow



Step 4 Drift score compared against configured threshold (`config.yaml`)



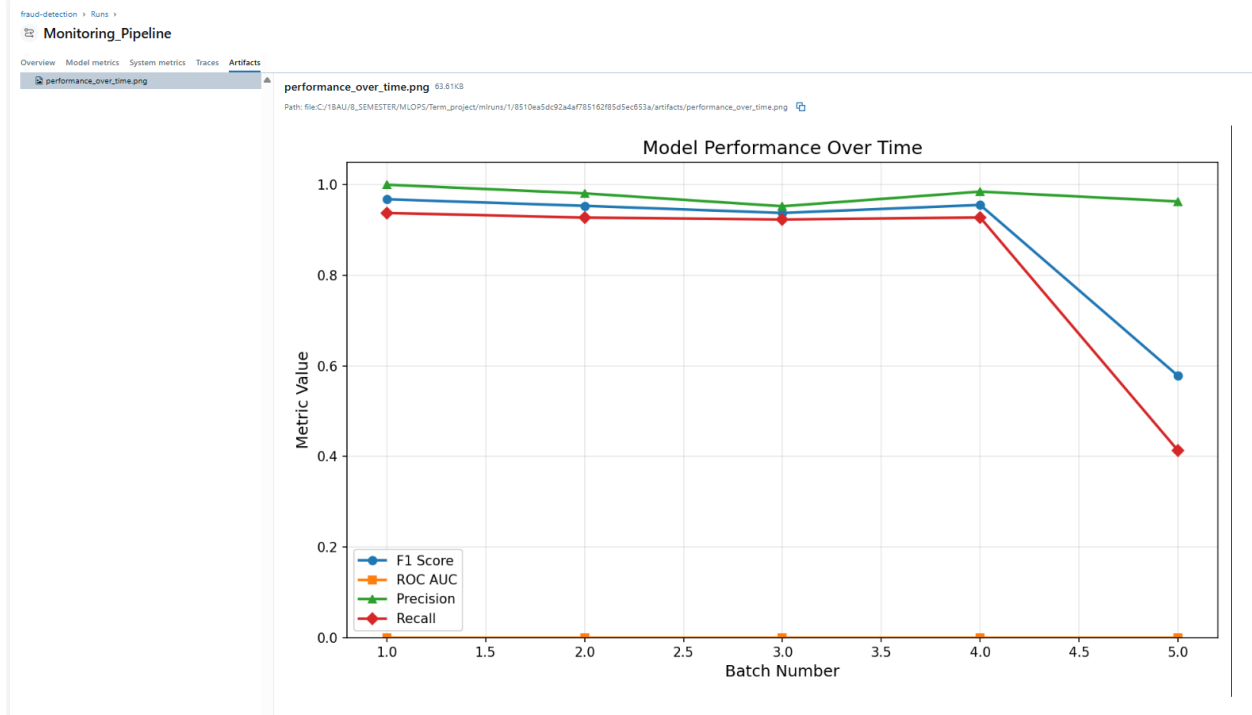
Step 5 If drift > threshold — trigger `src/auto_retrain.py`



Step 6 New model trained, evaluated, registered, and promoted to Production

Evidence 7: Data Drift Monitoring — Evidently AI Report

Evidently AI Drift Report saved as an HTML artifact inside MLflow Tracking UI, triggering auto-retraining logic.



Project Objectives Summary

Objective	Tool / Component	Evidence	Section
Experiment Tracking	MLflow Tracking + src/train.py	Evidence 1 & 2	Section 2
Hyperparameter Tuning	Optuna + Nested MLflow Runs	Evidence 3	Section 3
Model Registry	MLflow Model Registry (FraudDetector)	Evidence 4 & 5	Section 4
Model Deployment	FastAPI + MLflow URI	Evidence 6	Section 5
Drift Monitoring	Evidently AI + Auto-Retraining	Evidence 7	Section 6